

Reviewer Pack — Real Radical Degree Lower Bound for AC^0 Circuits Computing P...

Entry 6554531eadaa · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 12:40:43 UTC

Real Radical Degree Lower Bound for AC^0 Circuits Computing PARITY

Entry ID: 6554531eadaa **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 12:40:43 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): AC^0 Circuit Size for PARITY

Statement:

For any AC^0 circuit C computing PARITY on n inputs, the real radical of the ideal generated by the polynomial equations defining C has degree $\Omega(2^{\{n/2\}})$

Rationale (proposer's reasoning):

The real radical captures the essential structure of the polynomial equations, and its degree reflects the complexity required to represent PARITY, which AC^0 circuits cannot efficiently compute

Taxonomy category: AC^0_PARITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 88c3d6ae17c14b48

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_ac0_circuit(n):
        # Simplified AC0 circuit for parity function using XOR gates
        if n == 1:
            return [random.choice([0, 1])]
        else:
            left = generate_ac0_circuit(n // 2)
            right = generate_ac0_circuit(n - n // 2)
            return [left[i] ^ right[i % len(right)] for i in range(n)]

    def polynomial_from_circuit(circuit):
        # Convert circuit to a polynomial equation
        if len(circuit) == 1:
            return circuit[0]
        else:
            left = polynomial_from_circuit(circuit[:len(circuit) // 2])
            right = polynomial_from_circuit(circuit[len(circuit) // 2:])
            return left ^ right

    def real_radical_degree(poly):
        # Simplified method to estimate the degree of the real radical
        if isinstance(poly, int):
            return 0
        else:
            return max(real_radical_degree(poly[0]), real_radical_degree(poly[1])) + 1

    n = random.randint(5, 40)
    circuit = generate_ac0_circuit(n)
    poly = polynomial_from_circuit(circuit)

    degree = real_radical_degree(poly)
    lower_bound = math.ceil(2 ** (n / 2))

    conjecture_holds = degree >= lower_bound
    counterexample = "" if conjecture_holds else f"Degree {degree} < {lower_bound}"

    return {
        "metric_name": "Real Radical Degree",
        "metric_value": degree,
        "instances_tested": 1,
```

```

        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value:.2f} std={std_value:.2f} support_fraction={support_fraction:.2f}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"degree too low\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66b6d873.py", line 70, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66b6d873.py", line 47, in run_trial
    circuit = generate_ac0_circuit(n)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66b6d873.py", line 26, in generate_ac0_circuit
    left = generate_ac0_circuit(n // 2)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66b6d873.py", line 26, in generate_ac0_circuit
    left = generate_ac0_circuit(n // 2)
           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66b6d873.py", line 26, in generate_ac0_circuit
    left = generate_ac0_circuit(n // 2)
           ~~~~~^
[Previous line repeated 1 more time]
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_66b6d873.py", line 28, in generate_ac0_circuit
    return [left[i] ^ right[i % len(right)] for i in range(n)]
           ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to infinite recursion and IndexError, preventing data collection. The error suggests a flaw in the circuit generation code rather than directly refuting the conjecture. | next: Fix the generate_ac0_circuit function to handle base cases (e.g., n=0) and test with small n values to resolve recursion errors

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	101378
2	preregistration	ollama_remote	qwen3:8b	0	0	23927
3	novelty	ollama_remote	qwen3:8b	0	0	20431
4	novelty	ollama_remote	qwen3:8b	0	0	15620
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16318
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8095
7	judge	ollama_remote	qwen3:8b	0	0	20600

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 206370 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/6554531eadaa.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6554531eadaa.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6554531eadaa.tar.gz` (if generated)